

Build Your First Library

The path to PyPI

01

Function

pandas
code

02

Package

one folder

03

Build

make a
wheel

04

Publish

pip install

Today's goals

01 Package

Create a real, importable Python package.

02 Publish

Upload it to TestPyPI, then PyPI.

03 Install

`pip install` your own package.



**Build tiny. Install locally.
Publish safely.**



What we're building

```
import pandas as pd
import simple_eda as eda

df = pd.read_csv("data.csv")
eda.summarize(df)
eda.missing(df)
```

What we build now — and skip

Build now

One dependency: pandas

Plain Python functions

A clear README

Skip for now

CI/CD

Testing

Many dependencies



01

What is a Python library?



Library

Reusable code you install once and import anywhere.

Script vs library

Script

`python analyze.py`

Less convenient

Limited scope

Library

`import simple_eda`

More convenient

Broad scope

A tiny library is still real

No web app required.

No dashboard required.

Just useful, installable, importable.

The five bare minimums

A unique package name

A project folder

An `__init__.py` file

Your functions

Metadata — `pyproject.toml`



02

Design the smallest API



API

The set of functions users call. For example:
`eda.summarize(df)`, `eda.missing(df)`.



MVP

Minimum viable product — the smallest version that is genuinely useful, installable, and importable.



the simplest API

```
import simple_eda as eda  
eda.summarize(df)
```

Pick boring function names

Prefer (boring)

summarize

missing

numeric_columns

Avoid (clever)

hyper_profile_engine

magic_diagnose

mega_summary

MVP Input: Pandas dataframe only

Support pandas DataFrames first.

Not Polars, Spark, Dask, or Arrow yet.

Fewer choices means fewer bugs.



MVP Output: Plain objects only

```
{  
  "rows": 100,  
  "columns": 5,  
  "column_names": ["city", "sales"],  
}
```

The 0.1.0 mindset

First working version, not perfect.

Installable and understandable.

Momentum, not completeness.



03

Create the package folder

Final folder shape (src layout)

```
simple-eda-project/  
  src/  
    simple_eda/  
      __init__.py  
      core.py  
    README.md  
    pyproject.toml  
    LICENSE
```

Install name vs import name

Install name

simple-eda-you

hyphens

used by pip & PyPI

Import name

simple_eda

underscores

used in Python code



`__init__.py`

The file that marks a folder as an importable package and sets its top-level API — it is what lets users write ``import simple_eda``.

Why the src/ layout?

Blocks accidental root imports.

Matches modern Python projects.

Makes packaging behavior clear.

Common folder mistakes

pyproject.toml in the wrong folder.

Forgetting `__init__.py`.

Folder named `simple-eda`, not `simple_eda`.

Installing from the wrong directory.



04

Write the EDA functions



summarize()

```
def summarize(df):  
    return {  
        "rows": len(df),  
        "columns": len(df.columns),  
        "names": list(df.columns),  
    }
```

The core functions

summarize — shape, columns, types

missing — nulls per column

numeric_columns / categorical_columns



missing()

```
def missing(df):  
    counts = df.isna().sum()  
    return counts.sort_values(  
        ascending=False)
```

numeric & categorical columns

```
def numeric_columns(df):  
    sel = df.select_dtypes(  
        include="number")  
    return list(sel.columns)
```

```
def categorical_columns(df):  
    sel = df.select_dtypes(  
        include=["object", "category"])  
    return list(sel.columns)
```

Expose the public API

```
# src/simple_eda/__init__.py
from .core import (
    summarize,
    missing,
    numeric_columns,
    categorical_columns,
)
# __all__ = [...] lists the public names
```



Try it on a tiny DataFrame

```
import simple_eda as eda
```

```
df = pd.DataFrame({  
    "name": ["Ana", "Bo", None],  
    "age": [25, None, 31],  
})  
print(eda.summarize(df))  
print(eda.missing(df))
```

Function do's and don'ts

Do

Return data

Use plain dicts / Series

Keep signatures small

One job per function

Don't

Change df in place

Print instead of return

Add many optional args

One function for everything



05

Add pyproject.toml

What pyproject.toml does

Names your package and version.

Lists dependencies.

Tells build tools how to build.

Replaces old setup.py workflows.

A minimal pyproject.toml

```
[build-system]
```

```
requires = ["setuptools>=61"]
```

```
build-backend = "setuptools.build_meta"
```

```
[project]
```

```
name = "simple-eda-yourname"
```

```
version = "0.1.0"
```

```
description = "Tiny EDA helper"
```

```
dependencies = ["pandas>=1.5"]
```

Version numbers

Version

Means

0.1.0

First working version

0.1.1

A small fix

0.2.0

A new feature

1.0.0

Stable and public

README: install & use

```
# simple-eda-yourname
```

Tiny EDA helper for pandas.

```
## Install
```

```
pip install simple-eda-yourname
```

```
## Use
```

```
import simple_eda as eda  
print(eda.summarize(df))
```

License basics

Says what others may do with your code.

MIT is common for small projects.

Use an official template.

Never invent your own license text.



06

Install locally with pip

Install in editable mode

```
# run from the folder with pyproject.toml  
python -m pip install -e .
```

What -e (editable) means

Links pip to your source folder.

Code edits apply instantly.

No reinstall while developing.

Smoke-test the import

```
>>> import simple_eda as eda  
>>> eda.__all__  
['summarize', 'missing', ...]
```

When the import fails

Symptom

ModuleNotFoundError

Wrong folder

Wrong environment

Fix

Run `pip install -e .`

`cd` to the project root

Use `python -m pip`



07

Build a real package file

Build the artifacts

```
python -m pip install build
```

```
python -m build
```

The dist/ folder

dist/

■■■ simple_eda_yourname0.1.0.tar.gz

■■■ simple_eda_yourname0.1.0-...whl

Wheel vs source distribution

Source (.tar.gz)

A source archive

Used as a fallback

Wheel (.whl)

Prebuilt, fast install

What most users get

Keep build output out of git

dist/, build/, *.egg-info are generated.

Add them to .gitignore.

Bump the version before rebuilding.



08

Publish safely

Why TestPyPI first?

A practice package index.

Rehearse upload and install commands.

Mistakes are cheap here.

The upload flow

01 Check

```
python -m twine check dist/*
```

02 TestPyPI

```
twine upload --repository testpypi dist/*
```

03 PyPI

```
twine upload dist/* — only after TestPyPI works.
```

Install from TestPyPI

```
# install from TestPyPI
```

```
pip install \  
    -i https://test.pypi.org/simple/ \  
    --extra-index-url \  
        https://pypi.org/simple/ \  
simple-edu-yourname
```

Publishing safety

Use API tokens, not passwords.

Never commit tokens or secrets.

Fewer deps, less attack surface.

Beware typosquatting.



09

Use, improve & share

What to add — and avoid

Avoid early

Full HTML reports

Plugin systems

Every dataframe backend

AI-generated analysis

Good next steps

`quick_report()`

`top_categories()`

`column_summary()`

`duplicate_rows()`



Add a tiny test

```
def test_numeric_columns():  
    df = pd.DataFrame(  
        {"name": ["A"], "age": [10]})  
    cols = eda.numeric_columns(df)  
    assert cols == ["age"]
```

Final checklist

Package files all exist.

`pip install -e .` works.

`python -m build` works.

`twine check dist/*` passes.

README has install + usage.

0.1.0



Ship your first version

Four functions, importable, published.